



ADVANTEST CORPORATION

SmartShell 7 System Reference

User Guide

2023-08-28

Notices

Legal Notices

All rights reserved. All text and figures included in this publication are the exclusive property of Advantest Corporation. Reproduction of this publication in any manner without the written permission of Advantest Corporation is prohibited. Information in this document is subject to change without notice.

Trademarks and Registered Trademarks

- ADVANTEST is a trademark of Advantest Corporation.
- All other marks referenced herein are trademarks or registered trademarks of their respective owners.

Warranty

The material contained in this document is provided "as is," and is subject to being changed, without notice, in future editions. Further, to the maximum extent permitted by applicable law, Advantest disclaims all warranties, either express or implied, with regard to this material and any information contained herein, including but not limited to the implied warranties of merchantability and fitness for a particular purpose. Advantest shall not be liable for errors or for incidental or consequential damages in connection with the furnishing, use, or performance of this material or of any information contained herein. Should Advantest and the user have a separate written agreement with warranty terms covering the material in this document that conflict with these terms, the warranty terms in the separate agreement shall control.

Technology Licenses

The hardware and/or software described in this document are furnished under a license and may be used or copied only in accordance with the terms of such license.

Restricted Rights Legend

If software is for use in the performance of a U.S. Government prime contract or subcontract, Software is delivered and licensed as "Commercial computer software" as defined in DFAR 252.227-7014 (June 1995), or as a "commercial item" as defined in FAR 2.101(a) or as "Restricted computer software" as defined in FAR 52.227-19 (June 1987) or any equivalent agency regulation or contract clause. Use, duplication or disclosure of Software is subject to Advantest's standard commercial license terms for 93000 systems and non-DOD Departments and Agencies of the U.S. Government will receive no greater than Restricted Rights as defined in FAR 52.227-19(c)(1-2) (June 1987). U.S. Government users will receive no greater than Limited Rights as defined in FAR 52.227-14 (June 1987) or DFAR 252.227-7015 (b)(2) (November 1995), as applicable in any technical data.

Safety Notices

CAUTION

A CAUTION notice denotes a hazard. It calls attention to an operating procedure, practice, or the like that, if not correctly performed or adhered to, could result in damage to the product or loss of important data. Do not proceed beyond a CAUTION notice until the indicated conditions are fully understood and met.

WARNING

A WARNING notice denotes a hazard. It calls attention to an operating procedure, practice, or the like that, if not correctly performed or adhered to, could result in death or severe injury. Do not proceed beyond a WARNING notice until the indicated conditions are fully understood and met.

Table of Contents

1	Introduction	5
2	Components in SmartShell Environment.....	6
2.1	Basic Components in the SmartShell Application	6
2.2	Support for Easy Connection in the SmartShell Application.....	6
2.3	Support for EDA Tools.....	7
2.4	SmartShell Client GUI.....	7
3	Getting Started.....	8
3.1	Installation Package	8
3.2	SmartShell Root Path	8
3.3	SmartShell Server and Configuration.....	8
4	How to Launch the SmartShell Server	10
4.1	SmartShell Server Launch Option	10
4.2	SmartShell Server Modes.....	11
4.2.1	SmartShell Console Mode	11
4.2.2	SmartShell User Client Mode	11
4.2.3	SmartShell multi SmarTest session selection mode	12
4.3	Switching SmartShell Operation Modes and Exiting SmartShell	12
4.4	SmartShell Online Help Menu.....	12
5	The SmarTest Test Program Template.....	15
5.1	Test Program for Siemens ATE-Connect	15
6	The SmartShell Client Test Program	18
6.1	Configure Client Setup	18
6.2	SmartShell Client Test Script	18
6.2.1	Python Script.....	18
6.3	Configure Setup for Siemens ATE-Connect.....	20
7	Pattern Conversion and Execution using 3 rd Party Conversion Tool.....	22
7.1	STIL Pattern Conversion and Registration	22
7.2	Config STIL Pattern Conversion.....	24
7.2.1	Dynamically Specify STIL setup file	24
7.2.2	Streaming Setup File to SmartShell server.....	24

7.2.3	Conversion on the Tester Workstation	25
7.2.4	Conversion on a Different Workstation	26
8	Running Test via SmartShell.....	28
8.1	Running Test Directly in the SmartShell Server	28
8.2	Running the SmartShell Test Program Controlling SmarTest	28
8.3	Running EDA Tool Directly from EDA Test Script	29
8.3.1	Running Siemens SiliconInsight via Siemens ATE-Connect.....	29
9	SmartShell Client GUI.....	31
9.1	Introduction to SmartShell Client GUI	31
9.2	Overview of the SmartShell Client GUI	32
9.3	Config View	34
9.4	SmartShell Console View	35
9.5	Hypervisor View	36
9.6	Pattern Conversion View	37
9.7	Shmoo View	39

1 Introduction

SmartShell is a standard V93000 scripting interface to enable a test program or a test script other than a conventional V93000 SmartTest test program to control V93000 tester resources, execute the tests and retrieve the execution results. The SmartShell instructions and data are in standardized ASCII format whether they are stimulus or response.

SmartShell supports SmartTest 7 and SmartTest 8. It can link to EDA tools directly. SmartShell for SmartTest 7 supports digital pattern bring-up. SmartShell for SmartTest 8 supports both digital pattern bring-up and all other applications available on V93000 hardware. The command sets of SmartShell for SmartTest 7 and SmartTest 8 are nearly identical.

Note:

In this document, the term "test script" is used interchangeably with "test program" in the context of the SmartShell application, which is different from a conventional V93000 SmartTest test program to control the V93000 SOC tester.

2 Components in SmartShell Environment

2.1 Basic Components in the SmartShell Application

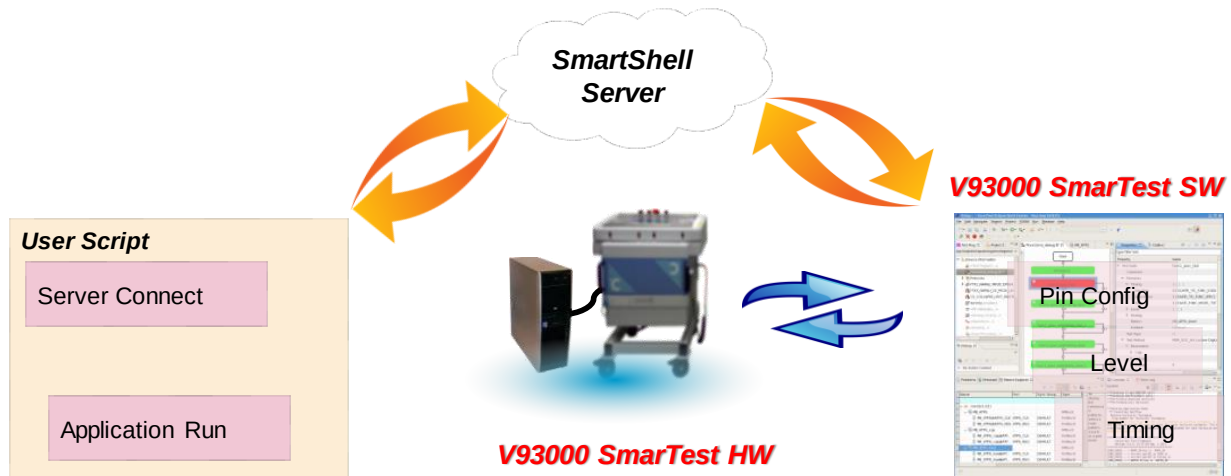


Fig. 1: Basic Components in SmartShell Environment

The basic components to run a SmartShell application in the SmartShell environment are:

- A ready-to-run SmarTest test program that:
 - Contains pin configuration, level and timing specifications, etc.
 - Has passed the continuity, leakage, power short tests.
 - Has passed functional tests for an integrity check of the device.
- A client test program or test script using the SmartShell commands. This client test program is called a SmartShell client test program.

2.2 Support for Easy Connection in the SmartShell Application

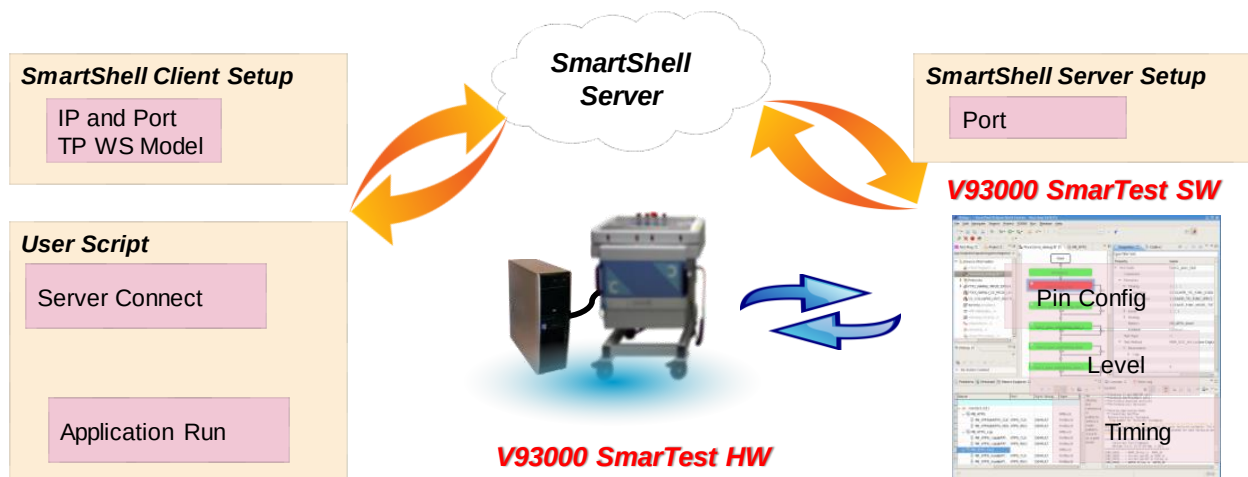


Fig. 2: Components in Enhanced SmartShell Environment

To make the connection easier, a SmartShell client test program can utilize the SmartShell client configuration file `smartshell_config.xml`. See details in the section 3.3 *SmartShell Server and Configuration*.

2.3 Support for EDA Tools

Beyond configurations in the SmartShell application, SmartShell also supports EDA tools such as Siemens Tessent SiliconInsight and Synopsys TestMax. The connection to Tessent SiliconInsight is established via Siemens ATE-Connect. The connection to TestMax is established via Synopsys ALE. In this way designer's program can be run directly from the EDA environment via SmartShell on a V93000 SOC tester.

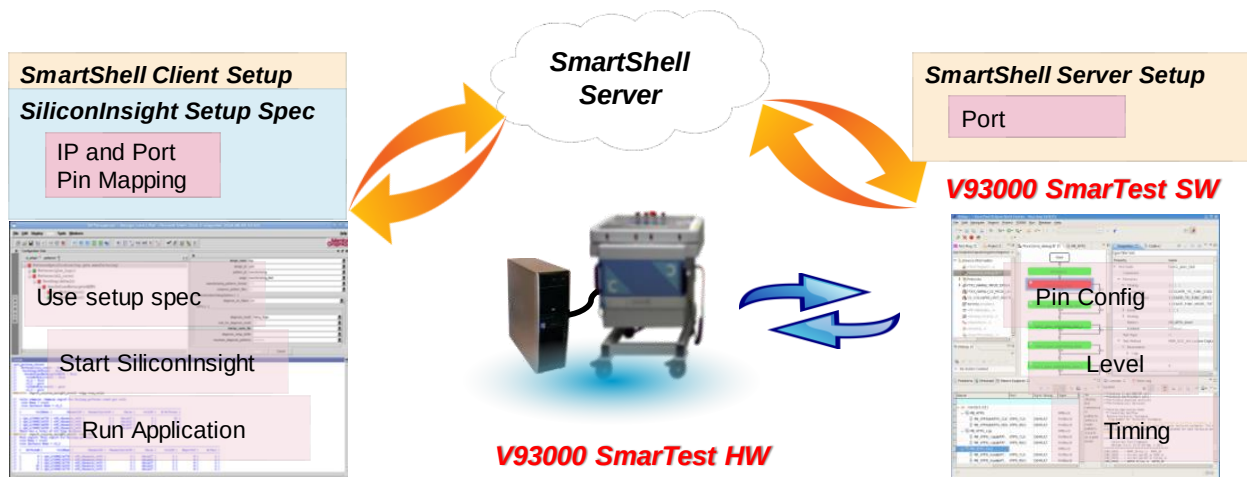


Fig. 3: Components in Siemens Tessent SiliconInsight Environment

Note:

For Siemens, the interface supports not only the IJTAG application and the pattern in Serial Vector Format (SVF) via the JTAG interface but also STIL ATPG pattern format. For details, refer to the Section **Pattern Conversion and Execution using 3rd Party Conversion Tool**.

2.4 SmartShell Client GUI

SmartShell offers a SmartShell Client GUI in which you can perform the following operations:

- Connect to an active SmartShell server running on a V93000 SmarTest tester workstation.
- Interactively control SmarTest with the SmartShell command set.
- Manage multiple test tasks in various programming languages for execution in line.
- Convert patterns in any pattern format with a customer-specified conversion tool.
- Add setups incrementally and dynamically to the SmarTest system for executing a test.
- Perform comprehensive shmoo tests for an entire testflow, test suites, or patterns.

For details, refer to the SmartShell reference document: **SmartShell for SmarTest 7 - SmartShell GUI**.

3 Getting Started

3.1 Installation Package

SmartShell for SmarTest 7 is installed through a standalone package: `v93000_smartshell-x.x.x.x-el#.x86_64.rpm`, where # stands for 5 (RHEL5) or 7 (EHEL7).

As root:

```
rpm --force -ivhU v93000_smartshell-x.x.x.x-el#.x86_64.rpm
```

where # stands for 5 (RHEL5) or 7 (EHEL7).

3.2 SmartShell Root Path

The SmartShell software installation folder is in

```
/opt/hp93000/smartshell64-x.x.x.x_el#
```

```
/opt/hp93000/smartshell -> /opt/hp93000/smartshell64-x.x.x.x_el#
```

3.3 SmartShell Server and Configuration

The SmartShell server and SmarTest run on the same workstation. The SmartShell server gets access to a SmarTest session through a following steps:

1. Launch SmarTest.
2. Load the test program.
3. Launch the SmartShell server.

Upon the launch of the SmartShell server, it looks up setups in the configuration file `config.xml` as shown in Fig. 4. After that, the SmartShell server waits for a client to connect to it for the SmartShell operation.

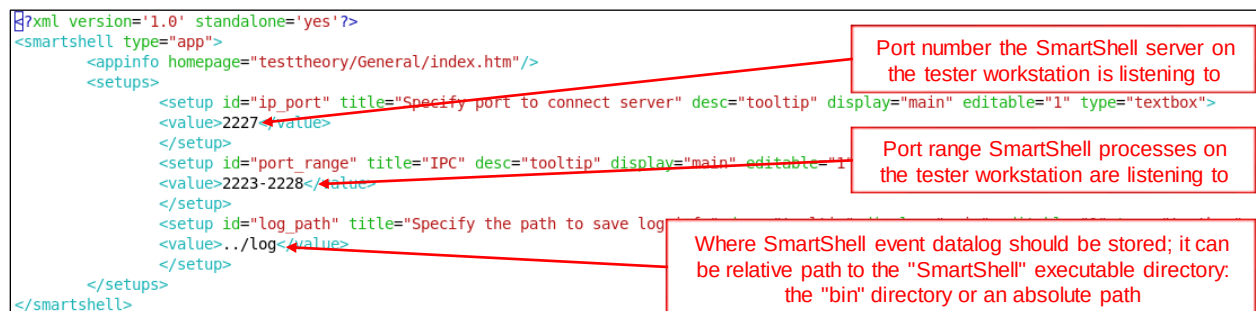


Fig. 4: SmartShell System "config.xml" File

The `ip_port` of 2227 is the default main port between the SmartShell server and user's SmartShell client test program. The folder specified for `log_path` stores temporary files which are deleted upon the closing of the SmartShell server. The log folder location can be changed.

The configuration file `config.xml` (in the folder `/opt/hp93000/soc/smartshell/config`) is editable.

It is a symbolic link to the folder `/var/opt/hp93000/soc_common/smartshell/config/`.

4 How to Launch the SmartShell Server

4.1 SmartShell Server Launch Option

The options of log details can be specified when the SmartShell server is launched:

```
/opt/hp93000/smartshell/bin/SmartShell <log options>
```

Log options are:

logcommand: log commands

logresult: log results

logprint: log both commands and results

All options can be used at the same time and be separated by a space.

The log file which logs the commands and the execution results is named as:

```
SmartShellEvent.log
```

If SmartShell crashes, the last 100 commands are logged in a log file named SmartShellDump.log.

The two log files mentioned above are stored in the folder /opt/hp93000/smartshell/log. This folder is a symbolic link to /var/opt/hp93000/soc_common/smartshell/log.

Once a new log file is generated, the older ones are renamed as <logfile>_YYYY-MM-DD.HH:MM:ss.

4.2 SmartShell Server Modes

There are several modes when launching the SmartShell server:

- SmartShell console mode
- SmartShell user client mode
- SmartShell multisession selection mode

4.2.1 SmartShell Console Mode

In this mode, you can type commands interactively in the SmartShell server console window at a command-prompt `SERVER >`.

To launch the SmartShell server in this mode, use the `scm` option:

```
/opt/hp93000/smartshell/bin/SmartShell scm <log options>
```

```
=====
==== SmartShell v3.4.4.4 - 20200305 ====
(c) 2017, Advantest Co., R&D Shanghai
=====
Connect to SmarTest session :1
Analyzing testflow A1_FT1...
Analyzing pin configuration ADV.cfg...
Analyzing timing A1.tim.mfh...
Initialization has succeeded
HINT: to break server mode, type "break":
SERVER >
```

Fig. 5: SmartShell Console Mode for Interactive Operation

4.2.2 SmartShell User Client Mode

In this mode, the SmartShell client test program connects to the SmartShell server and executes the tests.

To launch the SmartShell server in this mode, execute the following command:

```
/opt/hp93000/smartshell/bin/SmartShell <log options>
```

```
=====
==== SmartShell v3.4.4.4 - 20200305 ====
(c) 2017, Advantest Co., R&D Shanghai
=====
Connect to SmarTest session :1
Analyzing testflow A1_FT1...
Analyzing pin configuration ADV.cfg...
Analyzing timing A1.tim.mfh...
Initialization has succeeded
#Waiting for client to connect...
```

Fig. 6: SmartShell User Client Mode

4.2.3 SmartShell Multisession Selection mode

In this mode, the SmartShell server checks running SmartTest sessions first. If there is any, you can choose which SmartTest session to connect. If no SmartTest session is running, idle mode is activated, you can use the command "INIT" to start a new SmartTest session, or connect to a SmartTest session which is started after the SmartShell server was launched.

To launch the SmartShell server in this mode, use the `nac` option:

`nac`: SmartTest is not automatically connected.

`/opt/hp93000/smartshell/bin/SmartShell nac <log options>`

```
=====
=== SmartShell v3.6.0.0 - beta2 - 20230810 ===
(c) 2017, Advantest Co., R&D Shanghai
=====
There is 1 SmartTest session running:
session1 - :6
select a session, 0 means skip to choose a SmartTest session
[0..1]? :1
Choose session :6
```

Fig. 7: SmartShell Multisession Selection Mode

4.3 Switching SmartShell Operation Modes and Exiting SmartShell

To switch from the SmartShell user client mode to the SmartShell console mode, execute the following command in a Unix shell console window:

`/opt/hp93000/soc/smartshell/bin/to_server_mode`

To switch from the SmartShell console mode to the SmartShell user client mode, enter the command `break` at the server prompt `SERVER >`:

```
SERVER > break
```

To exit SmartShell, either press `Ctrl+c` or enter `quit` or `exit` in the SmartShell console window

`SERVER >`, or execute the command line `/opt/hp93000/soc/smartshell/bin/killsms` in a Unix shell console window.

4.4 SmartShell Online Help Menu

In the SmartShell server console window, once the SmartShell prompt `SERVER >` appears, you can directly issue SmartShell commands. In the SmartShell user client mode, the SmartShell prompt `SERVER >` appears only during the execution of the SmartShell client test program.

To learn the SmartShell commands, use the SmartShell online help menu with the command `HELP`.

```

SERVER > HELP
HELP
CHK          CLOSE_SMT  COPY          DELETE       EDIT         ENABLE       EXE          GET
HELP        HOLD       INIT         INSERT       SET          SHELL       SPLIT       USE

```

Fig. 8: Online Help Menu to Get List of Commands

The commands that are regularly used are SET, GET, and EXE. To get a list of commands starting with GET, enter a help command like this:

```
SERVER > HELP GET
```

All matched commands are shown.

```

SERVER > HELP GET
HELP GET
GET ALL_SESSIONS:      GET ALL_SESSIONS
GET EDGE:              GET EDGE hexnum,n
GET FALC TAG:          GET [FALC] [FALC_ONF] [FALC_PEND] [TAG] [hexlist] [STARTC=hexnum] [END_OF_CYCLE=hexnum]
GET FALC:              GET [FALC] [FALC_ONF] [FALC_PEND] [hexlist] [STARTC=hexnum] [END_OF_CYCLE=hexnum]
GET FLOW STATE_LIST:   GET FLOW [iflow] STATE_LIST
GET LEV:              GET LEV hexnum,n
GET PAT PATH:         GET PAT PATH
GET PAT PATH=hexnum:  GET PAT PATH=hexnum
GET PAT PRIORITY:      GET PAT PRIORITY
GET PAT SIGNAL_LIST:   GET PAT <label> SIGNAL_LIST
GET PAT TOTAL_CHC:     GET PAT <label> TOTAL_CHC
GET PAT TOTAL_VEC:     GET PAT <label> TOTAL_VEC
GET PAT VEC SIGNAL:    GET PAT <label> VEC hexnum restart stop
GET PAT VEC:           GET PAT <label> VEC restart
GET PAT VEC_REPEAT:    GET PAT <label> VEC_REPEAT vec_num
GET PAT [hex] LABEL_LIST: GET PAT [hex] LABEL_LIST

```

Fig. 9: Online Help Menu to Get List of Commands Starting with GET

If you want to learn how to use the command GET PAT TOTAL_VEC, execute the following command:

```
SERVER > HELP GET PAT TOTAL_VEC
```

```

SERVER > HELP GET PAT TOTAL_VEC
HELP GET PAT TOTAL_VEC
NAME:
    GET PAT TOTAL_VEC

DESCRIPTION:
    The "GET PAT TOTAL_VEC" command gets total vector number in a given main label

SYNTAX:
    GET PAT <label> TOTAL_VEC

PARAMETERS:
    label : main label name

USAGE:
    GET PAT jtag_pattern TOTAL_VEC

RETURN:
    total vector number

```

Fig. 10: Online Help Menu to Show the Command Syntax

If you enter an unrecognized command, a SYNTAX ERROR message is shown and all commands containing the entered string are listed. For example, if you enter GET PAT, since it is incomplete, all commands starting with GET PAT are listed.

In addition, entering a SmartShell command behaves similarly like a normal text editor and it is named as the "presskey" command editor. For example:

```

SERVER > GET PAT
ERROR: SYNTAX ERROR: command "GET PAT" correct syntax is below
GET PAT LABEL_LIST MAIN_LABELS: GET PAT LABEL_LIST MAIN_LABELS [<port>]
GET PAT LABEL_LIST MULTI_PORT: GET PAT [<mpbburst>] LABEL_LIST MULTI_PORT
GET PAT LABEL_LIST SINGLE_PORT: GET PAT LABEL_LIST SINGLE_PORT
GET PAT LABEL_LIST: GET PAT <label> LABEL_LIST <port>
GET PAT PORT_NAME: GET PAT <label> PORT_NAME <pin>
GET PAT PRIMARY: GET PAT PRIMARY
GET PAT SIGNAL_LIST: GET PAT <label> SIGNAL_LIST
GET PAT TOTAL_CYC: GET PAT <label> TOTAL_CYC
GET PAT TOTAL_VEC: GET PAT <label> TOTAL_VEC
GET PAT VEC SIGNAL: GET PAT <label> VEC <pin> <start> <stop>
GET PAT VEC: GET PAT <label> VEC <start> [WAVEFORM_TABLE]

```

Fig. 11: Online Help Menu to Get List of Commands

- Press the arrow up and down key to display the entered command before and after the current one.
- Press Ctrl-r and Ctrl-s to search a command in the command history for reusing a command.
- Press ESC and press Backspace to delete the word before the cursor until it hits special tokens.
- Press ESC twice and press Backspace to delete the word before the cursor until space or other special tokens.
- Press Ctrl-z or Ctrl-u to undo an operation.

To see all these functions, enter `HELP SHELL` at the server prompt `SERVER >` as shown in Fig. .

DESCRIPTION:

The "SMARTSHELL" interactive interface can issue SmartShell native command directly without writing user script. It enables ease of use in the SmartShell native scripting environment. User can learn first how the command behaves. The commands can be used in any script language.

PARAMETERS:

```

Command History
(^)/(v) arrow key: get existing input; or cursor move up/down for multiple line input
ctrl-p           : get existing input the same as (^) arrow keys
ctrl-n           : get existing input the same as (v) arrow keys
PGUP/PGDN keys  : get existing input up and down
ctrl-r           : get existing input with reverse search of existing inputs
ctrl-s           : get existing input with forward search of existing inputs

Cursor
(^)/(v) arrow key: cursor move up/down for multiple line input; or get existing input
ctrl-a/home      : cursor move to line begin, or at begin of the same line
ctrl-e/end       : cursor move to line end; or at end of the same line
ctrl-home        : cursor move to begin of the whole input: first char of first line
ctrl-end         : cursor move to end of the whole input: last char of the last line
ctrl-f or (>)    : cursor move one char to the right
ctrl-b or (<)    : cursor move one char to the left
ctrl-> arrow     : cursor move one word to the right
ctrl-< arrow     : cursor move one word to the left

Insert
alt-insert       : insert new line

Delete
ctrl-d/delete    : delete this char
alt-d/backspace  : delete previous char
ctrl-h           : delete previous char
ctrl-k           : delete rest of string input to the left
alt-k            : delete rest of string input to the right
ctrl-w/esc delete: delete word to the right
alt-w            : delete word to the left
esc backspace    : delete word to the left
esc esc backspace: delete several words to the left
ctrl-c/ctrl-l    : delete the whole input line

Undo/Redo
ctrl-z/ctrl-u    : undo typing
ctrl-y           : redo typing

Refresh/Clear/Update
ctrl-g           : corrupted input refreshed again
ctrl-x           : clear the entire screen
ctrl-o           : update terminal size when console window size is changed

Server Console Mode
alt-q            : quit SmartShell
alt-c            : switch to client script mode

User Client Mode
ctrl-c           : quit SmartShell

```

Fig. 12: SmartShell Console "presskey" Editor

5 The SmarTest Test Program Template

A V93000 SmarTest test program for a SmartShell operation must contain pin configuration, level and timing specifications. In addition, it should have passed the continuity, leakage, and power short tests to ensure that the board and device match the loaded SmarTest test program and the device operates normally.

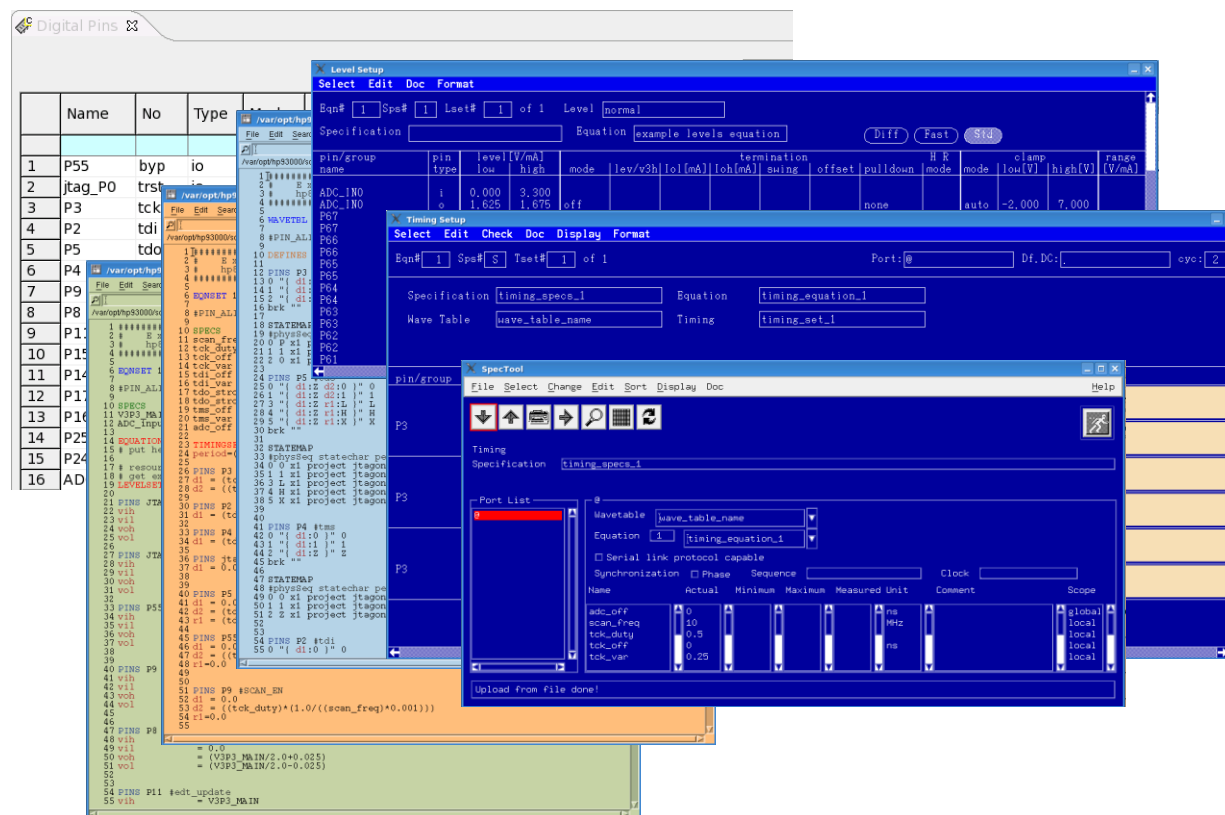


Fig. 13: User's SmarTest Test Program for SmartShell Operation

As soon as the SmarTest test program for the device under investigation is ready, the SmartShell operation can start.

5.1 Test Program for Siemens ATE-Connect

A multiport timing specification is needed for the Siemens-related tests. It must define a wavetable with a port containing five JTAG pins: tdo, tdi, tck, tms, trst, which are mapped with the JTAG protocol pins (see *Configure Setup for Siemens ATE-Connect* for details). The name of these pins can be customized.

The following figure is an example of the wavetable definition.

```

WAVETBL "wave_table_name" # This starts a wave table definition
DEFINES jtag_port test_port adc_port
PINS P3 #tck
0 "{ d1:1 d2:F0N }" P
1 "{ d1:1 }" 1
2 "{ d1:0 }" 0
brk ""
STATEMAP
#physSeq statechar period
0 P x1
1 1 x1
2 0 x1

PINS P5 #tdo
0 "{ d1:Z d2:0 }" 0
1 "{ d1:Z d2:1 }" 1
3 "{ d1:Z r1:L }" L
4 "{ d1:Z r1:H }" H
5 "{ d1:Z r1:X }" X
brk ""
STATEMAP
#physSeq statechar period
0 0 x1
1 1 x1
3 L x1
4 H x1
5 X x1

PINS P4 #tms
0 "{ d1:0 }" 0
1 "{ d1:1 }" 1
2 "{ d1:Z }" Z
brk ""
STATEMAP
#physSeq statechar period
0 0 x1
1 1 x1
2 Z x1

PINS P2 #tdi
0 "{ d1:0 }" 0
1 "{ d1:1 }" 1
2 "{ d1:Z }" Z
brk ""
STATEMAP
#physSeq statechar period
0 0 x1
1 1 x1
2 Z x1

PINS jtag_P0 #trst
0 "{ d1:0 }" 0
1 "{ d1:1 }" 1
2 "{ d1:Z }" Z
brk ""
STATEMAP
#physSeq statechar period
0 0 x1
1 1 x1
2 Z x1

```

Fig. 14: Timing Wavetable Example

The clock definition for tck in the wavetable needs to be with the state character P.

The wavetable definition for other ports such as test_port and adc_port is not shown in the figure, because they are not mandatory and can be renamed as desired.

The test suite which uses the multiport timing specification can be placed as the last test suite in the flow. In this way when the flow is executed by the SmartShell server, there is no need to change the primary timing.

In addition, you can also specify the timing specification with the SmartShell command `USE` which is introduced in detail in the command reference manual. Information on how to configure the interface is available in the section *Configure Setup for Siemens ATE-Connect*. See details on how to run the test program with ATE-Connect in the section *Running EDA Tool Directly from EDA Test Script*.

6 The SmartShell Client Test Program

The SmartShell package contains a Python library (for Python 2.7 and 3.7) for developing the SmartShell client test program in Python. They are in the following folder:

```
/opt/hp93000/smartshell/lib/python2.7
```

```
/opt/hp93000/smartshell/lib/python3.7
```

6.1 Configure Client Setup

For Python scripting language, the SmartShell client configuration file is `smartshell_config.xml` (Fig. 15) and it must be stored in your script directory. If `host` name is commented out, it is assumed that the user script will run from the same tester workstation where the SmartShell server is running.

```
<?xml version="1.0" encoding="UTF-8"?>
<smartshell_config>
  <!--if both ip and host is not defined, 127.0.0.1 is used as default ip.-->
  <host>10.150.110.182</host>
  <!--<host>sjclxmallard</host>-->
  <serverport>2227</serverport>
</smartshell_config>
```

Fig. 15: User "smartshell_config.xml" client setup file for controlling SmarTest externally

6.2 SmartShell Client Test Script

6.2.1 Python Script

In order to establish the link between the user script and SmartShell server, you need to add codes in the client Python test script to automatically initialize a SmartShell interface instance `shell_interface` between the client script and the SmartShell server.

There are two cases for how to add the initial code from the SmartShell Python library into the client Python test script:

- **When the client Python test script is located at the same tester workstation where SmarTest is running**

If you are using the default `smartshell_config.xml`, add this line to the python script:

```
sys.path.append('/opt/hp93000/smartshell/lib/python2.7')
```

If you are using a different `smartshell_config.xml`, add the following lines to the python script:

```
sys.path.append('/opt/hp93000/smartshell/lib/python2.7')
from spl_imports import *
import spl_globals
```

- **When the client Python test script is located at a different computer other than the tester workstation where SmartTest is running**

Copy /opt/hp93000/soc/smartshell/lib/python2.7 or /opt/hp93000/soc/smartshell/lib/python3.7 if you are using the python interpreter revision 2.7 or 3.7 respectively to the folder of the PC or the workstation where the client Python script is located. For example:

```
> scp <useraccount>@<smartshellhostname>:/opt/hp93000/soc/smartshell/lib/python2.7  
<smartshell python library folder>
```

Add the following lines of code into the client Python test script:

```
sys.path.append('<smartshell python library>/lib/python2.7')  
This will take the default smartshell_config.xml found in the client Python script  
folder.  
  
sys.path.append('<smartshell python library>/lib/python2.7')  
from spl_imports import *  
import spl_globals  
spl_globals.init('smartshell_config_online.xml')
```

The SmartShell client test program will read the client configuration file smartshell_config.xml or a different configuration file specified by the line of code spl_globals.init(<smartshell config file>) to connect to the SmartShell server.

To send the SmartShell commands to the server, use the following command:

```
shell_interface.sendCommand(<cmd string>)
```

Another Python function which has been introduced earlier is available:

```
shell_interface.generic_call(<cmd string>)
```

To get results, use the following line of code:

```
shell_interface.getResult()
```

To get the result right away use the following code:

```
test_results=shell_interface.sendCommand(<cmd string>)  
test_results=shell_interface.generic_call(<cmd string>)
```

6.3 Configure Setup for Siemens ATE-Connect

Working with Siemens SiliconInsight, you only need to adjust the following entries in the Siemens SiliconInsight EDA environment before running the client test program:

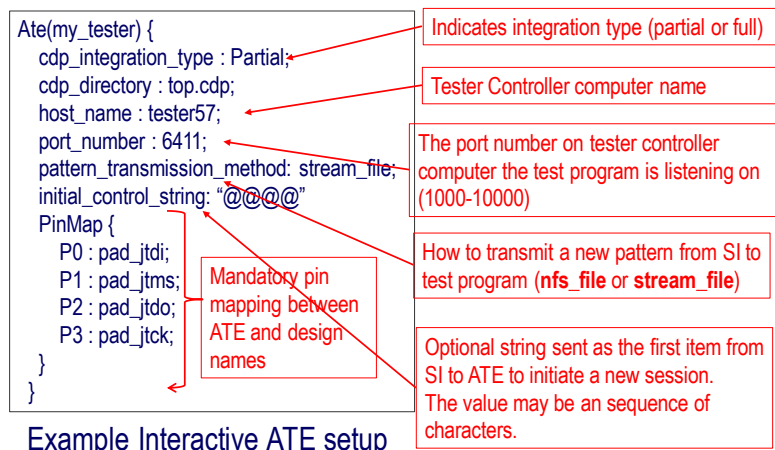


Fig. 16: Siemens SiliconInsight Configuration File "SiliconInsightSetupSpec.cfg"

You must define a correct pin-to-pin mapping. A complete setup specification file and how it is configured is shown in Fig. .

Only when the setup specification file is set up correctly (the correct name of tester workstation, the correct port of socket communication, and the correct pin mapping between design pins and SmartTest test program pins), the execution of the EDA works.

Siemens Tessent SiliconInsight software via ATE-Connect can perform the following operations:

- Creates test patterns based on the design rules specified by design engineers.
- Sends the pattern to SmartShell.
- Sends the measurement condition such as frequency and supply voltage to SmartShell.
- Sends the pattern modification instruction in the test pattern to SmartShell.
- Receives the test results from SmartShell.
- Processes the test results automatically within SiliconInsight software.
- Repeats this process by user's request or due to an exit condition defined by design engineers.

Most important commands are:

`ATEStreamPattern`, `ATEPatchSVFPattern/ATEExecutePatternGetSVFFailures` (for SVF pattern), `ATEPatchPattern/ATEExecutePatternGetFailures` (for STIL pattern).

For detailed command list and command descriptions, refer to the following Sections or contact Advantest SmartShell development team. And for more details on Siemens Tessent SiliconInsight, refer to Appendix A in the Tessent SiliconInsight User's Manual for Tessent Shell from Siemens.



Fig. 17: Siemens SiliconInsight Configuration File "SiliconInsightSetupSpec.cfg" example

7 Pattern Conversion and Execution using 3rd Party Conversion Tool

When SmartShell receives `ATEStreamPattern` command from SiliconInsight ATE-Connect, it will check the type of the received pattern stream. When the type is SVF pattern format, SmartShell will use the built-in conversion tool to convert the SVF pattern to the binl pattern file and download it to the system; when the type is STIL pattern format, it needs information such as what the STIL conversion tool is and where it is located, where the conversion shall take place: local to the same tester workstation or remote in the different workstation, and which option keyword is for the conversion as well as STIL conversion setup file. These information must be specified in the SmartShell configuration file `config.xml` located in the folder directory `/opt/hp93000/smartshell/config/`. From SmartShell 3.5.1.0 release onwards, SmartShell supports STIL pattern conversion with different STIL setup file using `CONV STILREADER SETUP` command.

Note:

The release 3.4.5.0 onwards supports `stilreader` or equivalent for STIL pattern conversion and `convmgr` or equivalent for entire test program conversion including STIL patterns defined in an xlsx spread sheet. `stilreader` and `convmgr` are from TestInsight.

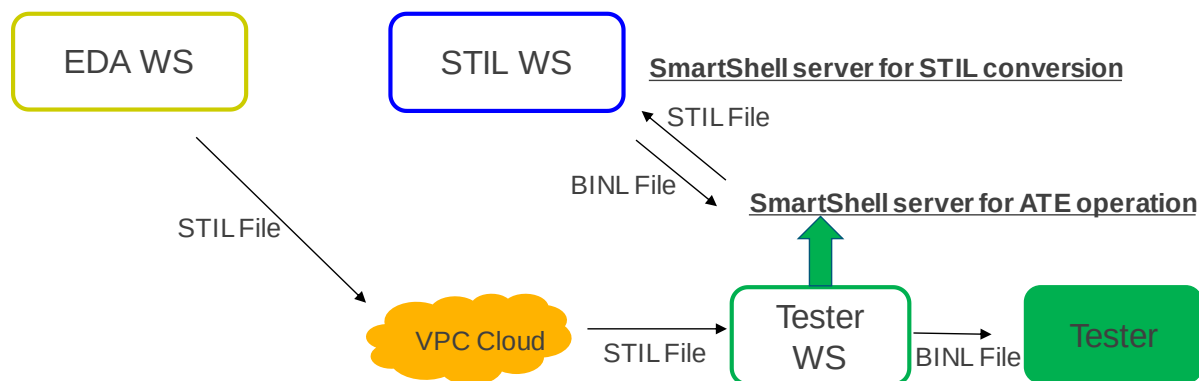


Fig. 7: SmartShell Pattern Conversion Process in a Different Workstation

7.1 STIL Pattern Conversion and Registration

The STIL pattern conversion happens via `ATEStreamPattern` command from the user's EDA script.

For SVF pattern:

```
ATEStreamPattern <svf pattern name>.svf @@ <complete pattern string in  
one trunk separated by @@ for new lines>
```

For STIL pattern:

```
ATEStreamPattern <stil pattern name>.stil @@ <complete pattern string  
in one trunk separated by @@ for new lines>
```

Where @@ is the token for the new line;

The SmartShell server will automatically register the converted and loaded STIL patterns and their correspondent burst label names and timing sets.

The relevant SmartShell commands for saving and using these converted STIL patterns are:

The "SAVE LOADED_STIL_PATTERN" command saves the list of STIL patterns which were converted and downloaded to SmarTest via ATEStreamPattern command to a csv file. This list is in csv format and each line contains a name of STIL pattern file, correspondent burst label name and timing set information. This csv file can be used for a SmartShell session at another point of time.

The "SET LOADED_STIL_PATTERN" command sets a list of STIL patterns which were converted and downloaded to SmarTest, so that the SmartShell knows which STIL patterns are converted and can inform these pre-loaded STIL patterns to the ATE-Connect. In this way ATE-Connect will not need to re-send the STIL patterns to the SmartShell for pattern conversion, instead it can execute the pattern immediately. This can save time and effort on an online tester time. This csv file can be generated by the SAVE LOADED_STIL_PATTERN mentioned above or manually created by the user.

The "GET LOADED_STIL_PATTERN" command return a list of preloaded STIL patterns which were converted and downloaded to SmarTest. This list can be used by the user script.

For command details, refer to the SmartShell command reference document.

Important Information when ATEStreamPattern command is used:

Original pin configuration setup may have additional pin groups added to the system for new timing setup. However original pin configuration file remains unchanged. When the user saves the timing setup after the conversion completes, to keep consistent setup across timing setup and pin configuration, the user will need to save the pin configuration in SmarTest Work Center or with SmartShell command:

```
SAVE SETUP PIN
```

Original pattern master file is not changed until either manually save it in SmarTest Work Center or with SmartShell command:

```
SAVE SETUP VECTOR
```

User shall consider to backup the original pattern master file in case unexpected happens.

Original timing master file is not changed instead SmartShell creates new timing master file preceded with __tmp_ to the original master file name as:

```
__tmp_<original_file>.mfh
```

This file can be used directly or be copied to a new file. The testflow can select this file or a new copied file for further operation.

For execution, the user will simply issue either `ATEExecutePatternGetSVFFailures` and `ATEExecutePatternGoNoGo` with SVF file name or `ATEExecutePatternGetFailures` and `ATEExecutePatternGoNoGo` with the STIL file name as their argument. Those commands are sent by Siemens SiliconInsight ATEConnect interface.

7.2 Config STIL Pattern Conversion

The `/opt/hp93000/smartshell/config/config.xml` is where the setup for conversion is defined:

Keyword	Option	Description
servertype	server	the server does SmartShell operation plus conversion
	converter	the server does only the conversion
	<host or ip>	specifies where the STIL conversion happens
stilconverter	<stil conversion tool>	specifies location of STIL conversion tool
	<option keyword>	specifies the keyword for conversion setup
	<stil setup file>	specifies the STIL setup file name
xlsxconverter	<convmgr conversion tool>	specifies location of convmgr tool

7.2.1 Dynamically Specify STIL setup file

For the situation where the conversion uses a different STIL setup file instead of the default one specified in the `config.xml` file, the command `CONV STILREADER SETUP` shall be used. Several pre-conditions need to be fulfilled when using this approach:

- 1) The STIL setup file and all associated setup files including default STIL pattern files must be accessible from the SmartShell server;
- 2) Environment variables required by STIL setup file must be set prior to the launch of the SmartShell server.

This command can be ideally used in a situation when the STIL setup file is a script.

Syntax:

```
CONV STILREADER SETUP=<stil setup file>
```

Return:

ERROR if file was not found

7.2.2 Streaming Setup File to SmartShell server

For the situation when the STIL setup file is a plain text file and all options are defined in it, the following two commands can be used to stream the setup file and default STIL pattern files to the SmartShell server:

CONV STLREADER STREAM SETUP: stream the complete setup file to the SmartShell.

The setup file may define several default STIL pattern with the keyword:

```
default_definitions = ['default stil file.stil', 'default stil file  
stil']
```

CONV STLREADER STREAM DEFAULT: to stream default STIL patterns to the SmartShell server

Syntax:

```
CONV STLREADER STREAM SETUP <stil_conversion.setup> @@ <complete setup  
file one trunk separated by @@ for new lines>
```

```
CONV STLREADER STREAM DEFAULTPAT <default stil file name>.stil @@  
<complete pattern string in one trunk separated by @@ for new lines>
```

Return:

pass

7.2.3 Conversion on the Tester Workstation

Edit /opt/hp93000/smartshell/config/config.xml

"servertype" is set to "server"

"stilconverter" is set to <stil conversion tool>

"xlsxconverter" is set to <convmgr conversion tool>

Can be different tool and different option code as well as different setup file

```
<?xml version='1.0' standalone='yes'?>
<smartshell type="app">
  <appinfo homepage="testtheory/General/index.htm"/>
  <setups>
    <setup id="ip_port" title="Specify port number the server is listening on" desc="tooltip" (
      <value>2227</value>
    </setup>
    <setup id="port_range" title="Specify port range the server internally uses" desc="tooltip"
    <setup id="log_path" title="Specify the path to save log data" desc="tooltip" display="main" edit
    <setup id="debug" title="Flag to enable logging verbose" desc="tooltip" display="main" edit
    <setup id="reload" title="Flag to enable vector reload when smartshell exits" desc="tooltip"
    <setup id="servertime" title="Specify the server type" desc="tooltip" display="main" edit
    <!--
      server:      this server does both SmartShell operation and conversion operation
      converter:   this server does only conversion operation
      host or ip:  the remote server at which the conversion is performed
                  in this case stilconverter and xlsxconvmgr entries are ignored
    -->
    <!-- <value>host</value> -->
    <value>server</value>
    <!-- <value>converter</value> -->
  </setup>
  <setup id="stilconverter" title="Specify stil conversion tool with the full path" desc="tool
    <value>/opt/93000/stdl/bin/stilreader</value>
    <value>-setup</value>
    <value>/opt/93000/stdl/config/stil.setup</value>
  </setup>
  <setup id="xlsxconvmgr" title="Specify convmgr tool with the full path" desc="tooltip" disp
    <value>/opt/93000/stdl/bin/convmgr</value>
  </setup>
</setups>
</smartshell>
```

7.2.4 Conversion on a Different Workstation

Edit /opt/hp93000/smartshell/config/config.xml

at tester workstation and STIL converter workstation

At tester workstation:

- "servertime" is set to "<converter host or ip>"

At STIL converter workstation

- "servertime" is set to "converter"
- "stilconverter" is set to <stil conversion tool>
- "xlsxconverter" is set to <convmgr conversion tool>
- And "ip_port" is set to 2229

```

<?xml version='1.0' standalone='yes'?>
<smartshell type="app">
  <appinfo homepage="testtheory/General/index.htm"/>
  <setup>
    <setup id="ip_port" title="Specify port number the server is listening on" desc="tooltip" d
      <value>2227</value>
    </setup>
    <setup id="port_range" title="Specify port range the server internally uses" desc="tooltip"
    <setup id="log_path" title="Specify the path to save log data" desc="tooltip" display="main"
    <setup id="debug" title="Flag to enable logging verbose" desc="tooltip" display="main" edit
    <setup id="reload" title="Flag to enable vector reload when smartshell exits" desc="tooltip"
    <setup id="servertype" title="Specify the server type" desc="tooltip" display="main" editab
    <!--
      server:      this server does both SmartShell operation and conversion operation
      converter:   this server does only conversion operation
      host or ip:  the remote server at which the conversion is performed
                  in this case stilconverter and xlsconvmgr entries are ignored
    -->
    <value>conversion_host_workstation_name</value>
    <!-- <value>server</value> -->
    <!-- <value>converter</value> -->
  </setup>
  <setup id="stilconverter" title="Specify stil conversion tool with the full path" desc="too
    <value>opt/93800/stdl/bin/stilreader</value>
    <value>-setup</value>
    <value>opt/93800/stdl/config/stil.setup</value>
  </setup>
  <setup id="xlsconvmgr" title="Specify convmgr tool with the full path" desc="tooltip" disp
    <value>opt/93800/stdl/bin/convmgr</value>
  </setup>
</setup>
</smartshell>

```

Ignored

Ignored

```

<?xml version='1.0' standalone='yes'?>
<smartshell type="app">
  <appinfo homepage="testtheory/General/index.htm"/>
  <setup>
    <setup id="ip_port" title="Specify port number the server is listening on" desc="tooltip" (
      <value>2229</value>
    </setup>
    <setup id="port_range" title="Specify port range the server internally uses" desc="tooltip"
    <setup id="log_path" title="Specify the path to save log data" desc="tooltip" display="mair
    <setup id="debug" title="Flag to enable logging verbose" desc="tooltip" display="main" edit
    <setup id="reload" title="Flag to enable vector reload when smartshell exits" desc="tooltip"
    <setup id="servertype" title="Specify the server type" desc="tooltip" display="main" editat
    <!--
      server:      this server does both SmartShell operation and conversion operation
      converter:   this server does only conversion operation
      host or ip:  the remote server at which the conversion is performed
                  in this case stilconverter and xlsconvmgr entries are ignored
    -->
    <!-- <value>conversion_host_workstation_name</value> -->
    <!-- <value>server</value> -->
    <value>converter</value>
  </setup>
  <setup id="stilconverter" title="Specify stil conversion tool with the full path" desc="too
    <value>opt/93800/stdl/bin/stilreader</value>
    <value>-setup</value>
    <value>opt/93800/stdl/config/stil.setup</value>
  </setup>
  <setup id="xlsconvmgr" title="Specify convmgr tool with the full path" desc="tooltip" disp
    <value>opt/93800/stdl/bin/convmgr</value>
  </setup>
</setup>
</smartshell>

```

8 Running Test via SmartShell

When there is a SmarTest test program available, you can run the test through SmartShell.

8.1 Running Test Directly in the SmartShell Server

1. Start the SmartShell server in SmartShell console mode.
2. In the SmartShell console window, enter commands to control the SmarTest for the SmartShell operation.

```
=====
==== SmartShell v3.4.4.4 - 20200305 ====
(c) 2017, Advantest Co., R&D Shanghai
=====
Connect to SmarTest session :1
Analyzing testflow A1_FT1...
Analyzing pin configuration ADV.cfg...
Analyzing timing A1.tim.mfh...
Initialization has succeeded
HINT: to break server mode, type "break":
SERVER >
```

Fig. 8: Starting SmartShell server and launch user's SmarTest test program

Fig. 8 shows the EXE FLOW command to run the testflow.

```
EXE FLOW
```

Return

```
MG_Test_1 P
MG_Test P
bin_disconnect P
```

8.2 Running the SmartShell Test Program Controlling SmarTest

Having followed the steps described in *The SmarTest Test Program Template* and in *The SmartShell Client Test Program*, you can now run the client SmartShell test program written in Python.

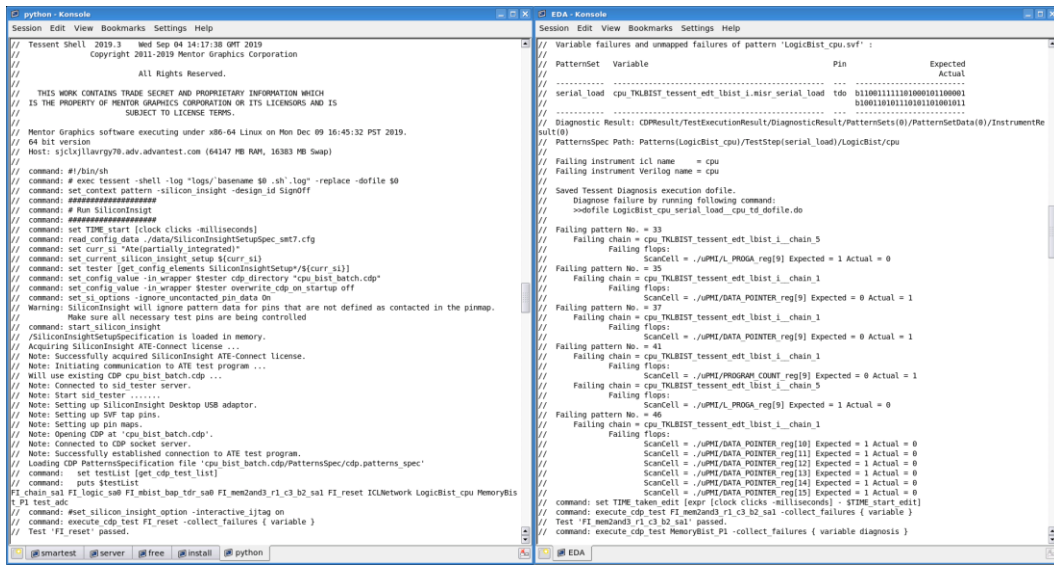
1. Start SmartShell in SmartShell user client mode.

```
=====
==== SmartShell v3.4.4.4 - 20200305 ====
(c) 2017, Advantest Co., R&D Shanghai
=====
Connect to SmarTest session :1
Analyzing testflow A1_FT1...
Analyzing pin configuration ADV.cfg...
Analyzing timing A1.tim.mfh...
Initialization has succeeded
#Waiting for client to connect...
```

Fig. 9: Launch SmartShell Server

2. Run the Python script to automatically connect to the SmartShell server and execute the tests.

2. Execute the Tcl script from the user's command console or from the EDA tool.



```
// Tesseract Shell 2019.3 Wed Sep 04 14:17:38 GMT 2019
// Copyright 2011-2019 Mentor Graphics Corporation
//
// All Rights Reserved.
//
// THIS WORK CONTAINS TRADE SECRET AND PROPRIETARY INFORMATION WHICH
// IS THE PROPERTY OF MENTOR GRAPHICS CORPORATION OR ITS LICENSORS AND IS
// SUBJECT TO LICENSE TERMS.
//
// Mentor Graphics software executing under x86-64 Linux on Mon Dec 09 16:45:32 PST 2019.
// 64 bit version
// Host: sjckajllavrgy70.advantest.com (64147 MB RAM, 16383 MB Swap)
//
// command: #!/bin/sh
// command: # exec tesseract -shell -log "logs/basename $0 .sh.log" -replace -dofile $0
// command: set context pattern -silicon_insight -design_id SigOff
// command: #####
// command: # Run SiliconInsight
// command: #####
// command: set TIME_start {clock clicks -milliseconds}
// command: read config_data ./data/SiliconInsightSetupSpec_sm7.cfg
// command: set curr_sl "Attepartially integrated"
// command: set current_silicon_insight_setup $(curr_sl)
// command: set tester {get config elements SiliconInsightSetup/$curr_sl}
// command: set config_value -in wrapper stester cdp directory "cpu_bist_batch.cdp"
// command: set config_value -in wrapper stester overwrite_cdp_on_startup off
// command: set sj_options -ignore unconnected pin data on
// Warning: SiliconInsight will ignore pattern data for pins that are not defined in the pimap.
// Make sure all necessary test pins are being controlled
// command: start_silicon_insight
// /SiliconInsightSetupSpecification is loaded in memory.
// Acquiring SiliconInsight ATE-connect license ...
// Note: Successfully acquired SiliconInsight ATE-Connect license.
// Note: Initiating communication to ATE test program ...
// Will use existing CDP cpu_bist_batch.cdp ...
// Note: Connected to sid tester server.
// Note: Start sid tester ...
// Note: Setting up SiliconInsight Desktop USB adaptor.
// Note: Setting up SWP tap pins.
// Note: Setting up pin maps.
// Note: Opening CDP at "cpu_bist_batch.cdp".
// Note: Connected to CDP socket server.
// Note: Successfully established connection to ATE test program.
// Loading CDP PatternsSpecification file "cpu_bist_batch.cdp/PatternsSpec/cdp_patterns_spec"
// command: set testlist {get cdp test list}
// command: puts testlist
// t chain sal fi_logic_saw fi_mist_bap_tdr_saw fi_mem2and3_r1_c3_b2_sal fi_reset ICLNetwork LogicBist_cpu MemoryBist
// fi test adc
// command: #set silicon_insight_option -interactive ijtag on
// command: execute cdp test fi_reset -collect_failures { variable }
// Test "fi_reset" passed.
//
// Variable Failures and unmapped Failures of pattern 'LogicBist_cpu.svf':
//
// PatternSet Variable Pin Expected Actual
// -----
// serial_load cpu_TNLRBIST_tesseract_edt_lbist_i_misr_serial_load tdo 011001111101000101100001
// b1001101011101010101001011
//
// Diagnostic Results: CDPResult/TestExecutionResult/DiagnosticResult/PatternSetData(0)/InstrumentRe
// sult(0)
//
// PatternsSpec Path: Patterns(LogicBist_cpu)/TestStep(serial_load)/LogicBist_cpu
//
// Failing instrument icl name = cpu
// Failing instrument Verilog name = cpu
//
// Saved Tesseract diagnosis execution dofile.
// Diagnose failure by running following command:
// >>dofile LogicBist_cpu_serial_load_cpu_tdr_dofile.do
//
// Failing pattern No. = 33
// Failing chain = cpu_TNLRBIST_tesseract_edt_lbist_i_chain_5
// Failing Flaps:
// ScanCell = ./UPM/L_PROGMA_reg[9] Expected = 1 Actual = 0
//
// Failing pattern No. = 35
// Failing chain = cpu_TNLRBIST_tesseract_edt_lbist_i_chain_1
// Failing Flaps:
// ScanCell = ./UPM/DATA_POINTER_reg[9] Expected = 0 Actual = 1
//
// Failing pattern No. = 37
// Failing chain = cpu_TNLRBIST_tesseract_edt_lbist_i_chain_1
// Failing Flaps:
// ScanCell = ./UPM/DATA_POINTER_reg[9] Expected = 0 Actual = 1
//
// Failing pattern No. = 41
// Failing chain = cpu_TNLRBIST_tesseract_edt_lbist_i_chain_1
// Failing Flaps:
// ScanCell = ./UPM/PROGRAM_COUNT_reg[9] Expected = 0 Actual = 1
//
// Failing chain = cpu_TNLRBIST_tesseract_edt_lbist_i_chain_5
// Failing Flaps:
// ScanCell = ./UPM/L_PROGMA_reg[9] Expected = 1 Actual = 0
//
// Failing pattern No. = 46
// Failing chain = cpu_TNLRBIST_tesseract_edt_lbist_i_chain_1
// Failing Flaps:
// ScanCell = ./UPM/DATA_POINTER_reg[10] Expected = 1 Actual = 0
// ScanCell = ./UPM/DATA_POINTER_reg[11] Expected = 1 Actual = 0
// ScanCell = ./UPM/DATA_POINTER_reg[12] Expected = 1 Actual = 0
// ScanCell = ./UPM/DATA_POINTER_reg[13] Expected = 1 Actual = 0
// ScanCell = ./UPM/DATA_POINTER_reg[14] Expected = 1 Actual = 0
// ScanCell = ./UPM/DATA_POINTER_reg[15] Expected = 1 Actual = 0
//
// command: set TIME_taken_edit {expr {clock clicks -milliseconds} - TIME_start_edit}
// command: execute cdp test fi_mem2and3_r1_c3_b2_sal -collect_failures { variable }
// Test "fi_mem2and3_r1_c3_b2_sal" passed.
// command: execute cdp test MemoryBist_P1 -collect_failures { variable diagnosis }
```

Fig. 12: Execution of Designer's EDA Script

9 SmartShell Client GUI

9.1 *Introduction to SmartShell Client GUI*

The SmartShell Client GUI can be run anywhere in the network to interactively control V93000 SmarTest via the SmartShell command set. The SmartShell Client GUI provides an easy-to-use interface to perform the following operations:

- Connect to an active SmartShell server running on a V93000 SmarTest tester workstation.
- Interactively control SmarTest with the SmartShell command set.
- Manage multiple test tasks in various programming languages for execution in line.
- Convert patterns in any pattern format with a customer-specified conversion tool.
- Add setups incrementally and dynamically to the SmarTest system for executing a test.
- Perform comprehensive shmoo tests for an entire testflow, test suites, or patterns.

To launch the SmartShell Client GUI, execute the following command:

```
/opt/hp93000/smartshell/gui/ssclient &
```

9.2 Overview of the SmartShell Client GUI

In the SmartShell Client GUI window, you can see the Configuration view, SmartShell Console view, Hypervisor view, Pattern Conversion view, Shmoo view, and Console view.

If you click a view tab, the main window appears.

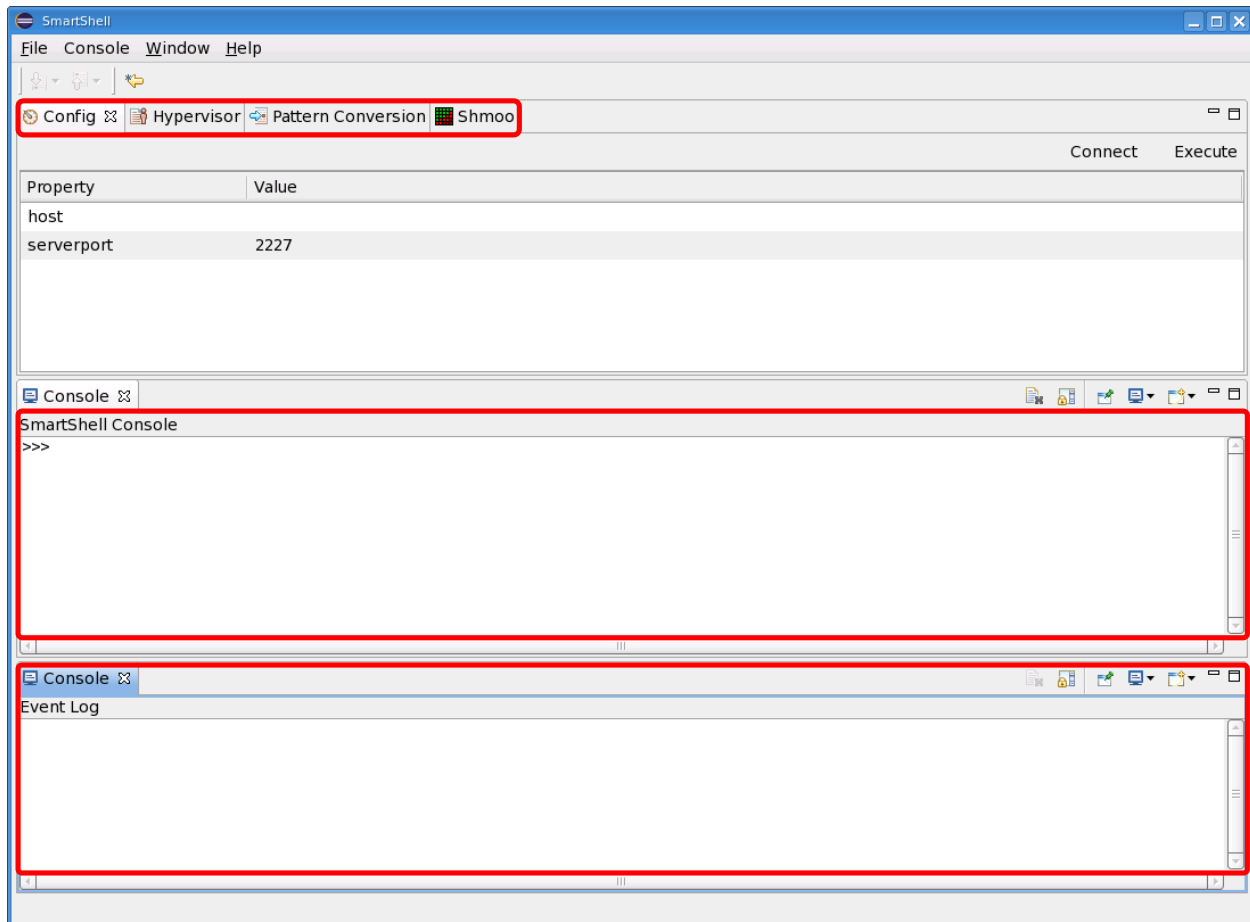


Fig. 13: SmartShell Client GUI Window

Config View

This menu is to configure the host and server port of the SmartShell server to which the SmartShell Client GUI connects.

SmartShell Console View

In the Console view, you can interactively issue SmartShell commands to the SmartShell server for the SmartShell operations.

Hypervisor View

This is a task scheduler to add the user-defined test programs or test scripts to be executed in the order listed. You can enter the email address to inform the owner of each test task when the execution is completed.

Pattern Conversion View

This view is for pattern conversion. Currently, the STIL format conversion using stilreader from TestInsight is supported.

Shmoo View

In Shmoo view, you can specify tester resources such as specification variables or hardware resources for the device characterization and debug.

Event Log Console View

It shows the execution messages.

9.3 Config View

Configure host and serverport, and then click the **Connect** button to connect to the SmartShell server.

Procedure

1. Open the SmartShell Client GUI.
2. Click the **Config** tab.
3. Setup **host** and **serverport**.
The **host** value can be left empty if the client GUI runs at the same workstation as the SmartShell server.
4. Click **Connect** to connect to the SmartShell server.

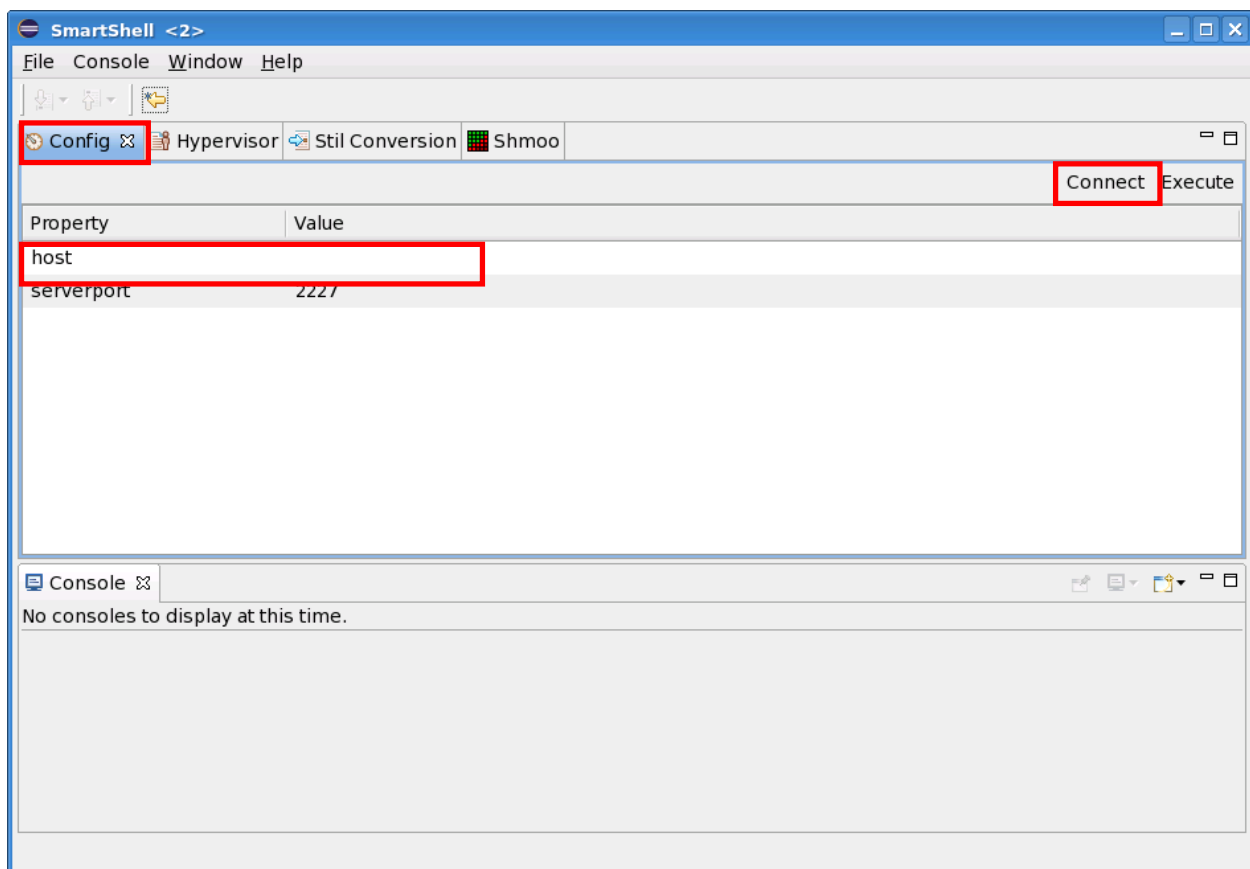


Fig. 14: SmartShell Client GUI Window – Config View

9.4 SmartShell Console View

You can directly enter SmartShell commands and execute tests without writing a client test script in the SmartShell Console view.

Procedure

1. Open the SmartShell Client GUI.
 2. Click the **Config** tab.
 3. Setup **host** and **serverport**.
 4. Click the **Connect** button to connect to the SmartShell server.
 5. Click the SmartShell **Console** tab.
- SmartShell commands can be issued and executed in this window.

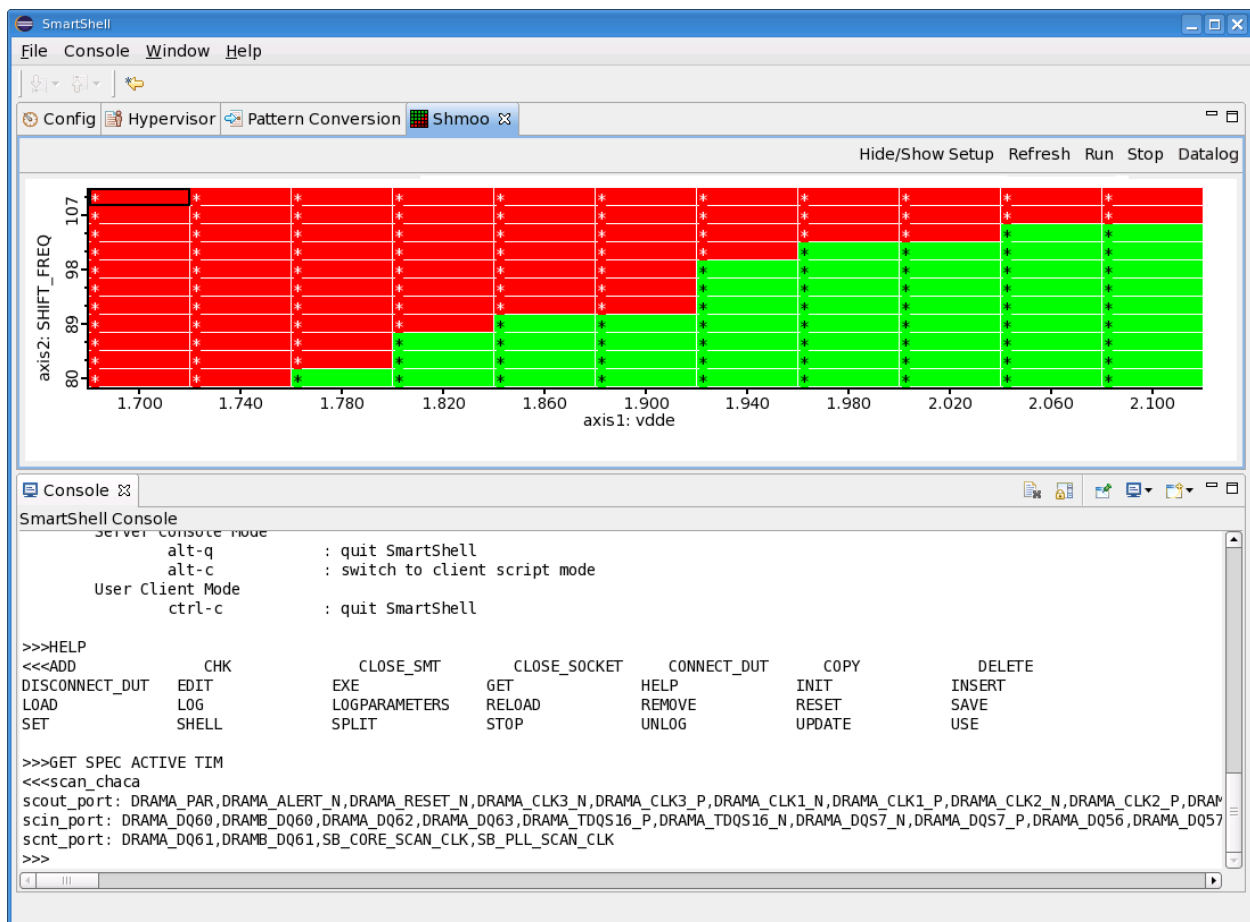


Fig. 15: SmartShell Client GUI Window – SmartShell Console View

Note:

Clicking the **Stop** button breaks the shmoo execution.
Datalog stores the shmoo result as a table in a file.

9.5 Hypervisor View

Hypervisor is a SmartShell Client GUI feature that is used to schedule and execute the client test program or test script in the order listed, and to send the execution status to the owner of each task.

Procedure:

1. Open the SmartShell Client GUI.
2. Click the **Config** tab.
3. Setup **host** and **serverport**.
4. Click the **Connect** button to connect to the SmartShell server.
5. Click the **Hypervisor** tab.
6. Click the **Add** button to add the scripts for execution.
7. Choose the script (see notes below) that you want to execute.
8. Click the **Execute** button.

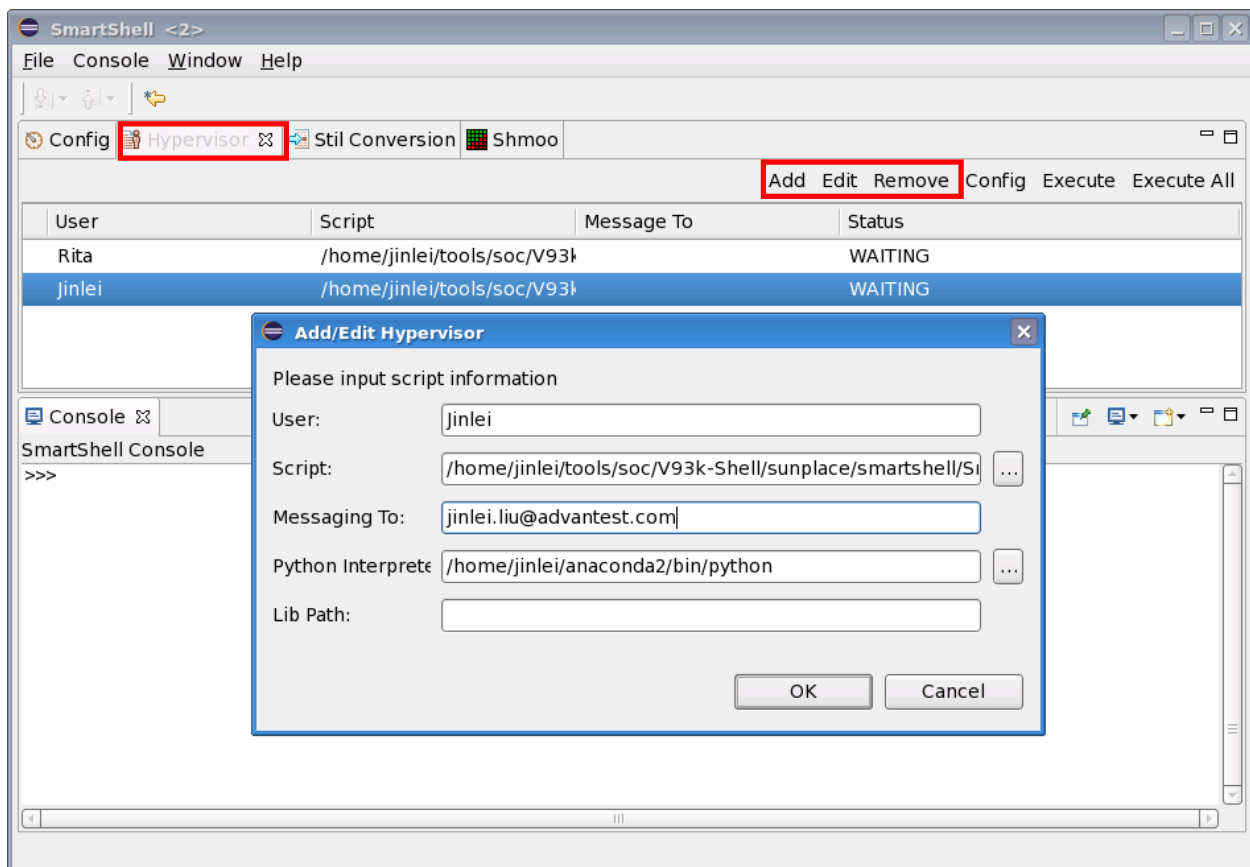


Fig. 16: SmartShell Client GUI Window – Hypervisor View

Note:

The script can be written in any scripting language such as Python, Tcl, or perl. It must be self-contained, which means this script can be executed with any other dependencies the SmartShell Client GUI needs to take care of.

9.6 Pattern Conversion View

Pattern conversion is a SmartShell Client GUI feature that can be used to convert pattern such as STIL patterns and execute the generated burst in SmarTest. This pattern conversion GUI supports *stilreader* for STIL pattern conversion and *convmgr* for spreadsheet-based test program conversion.

Procedure for STIL Pattern

1. Open the SmartShell Client GUI.
2. Click the **Config** tab.
3. Setup **host** and **serverport**.
4. Click the **Connect** button to connect to the SmartShell server.
5. Click the **Pattern Conversion** tab.
6. Select **STIL Conversion**.
7. Choose the conversion setup file in **Conversion Setup**.
8. Choose the Pattern file such as STIL in **Pattern file**.
9. Click the **Run** button.
10. Click **Run Pattern Shmoo** to go to the Shmoo view.

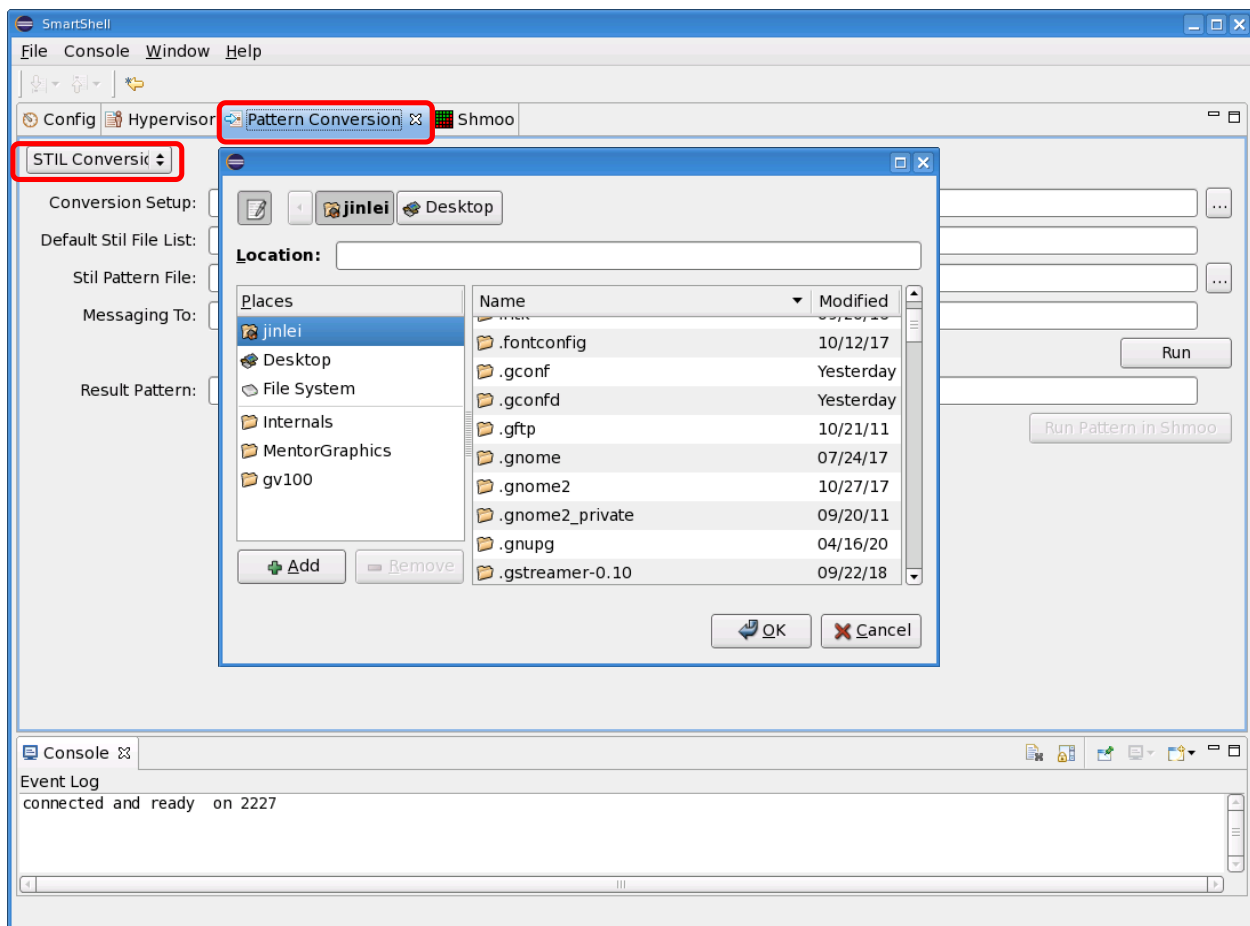


Fig. 17: SmartShell Client GUI Window – Pattern Conversion View

Procedure for Spreadsheet-Based Test Program Conversion

1. Open the SmartShell Client GUI.
2. Click the **Config** tab.
3. Setup **host** and **serverport**.
4. Click the **Connect** button to connect to the SmartShell server.
5. Click the **Pattern Conversion** tab.
6. Select **ConvManager**.
7. Choose the conversion setup file in **Spread File**.
8. Click the **Run** button.
9. Click the **Shmoo** tab to go to the Shmoo view.

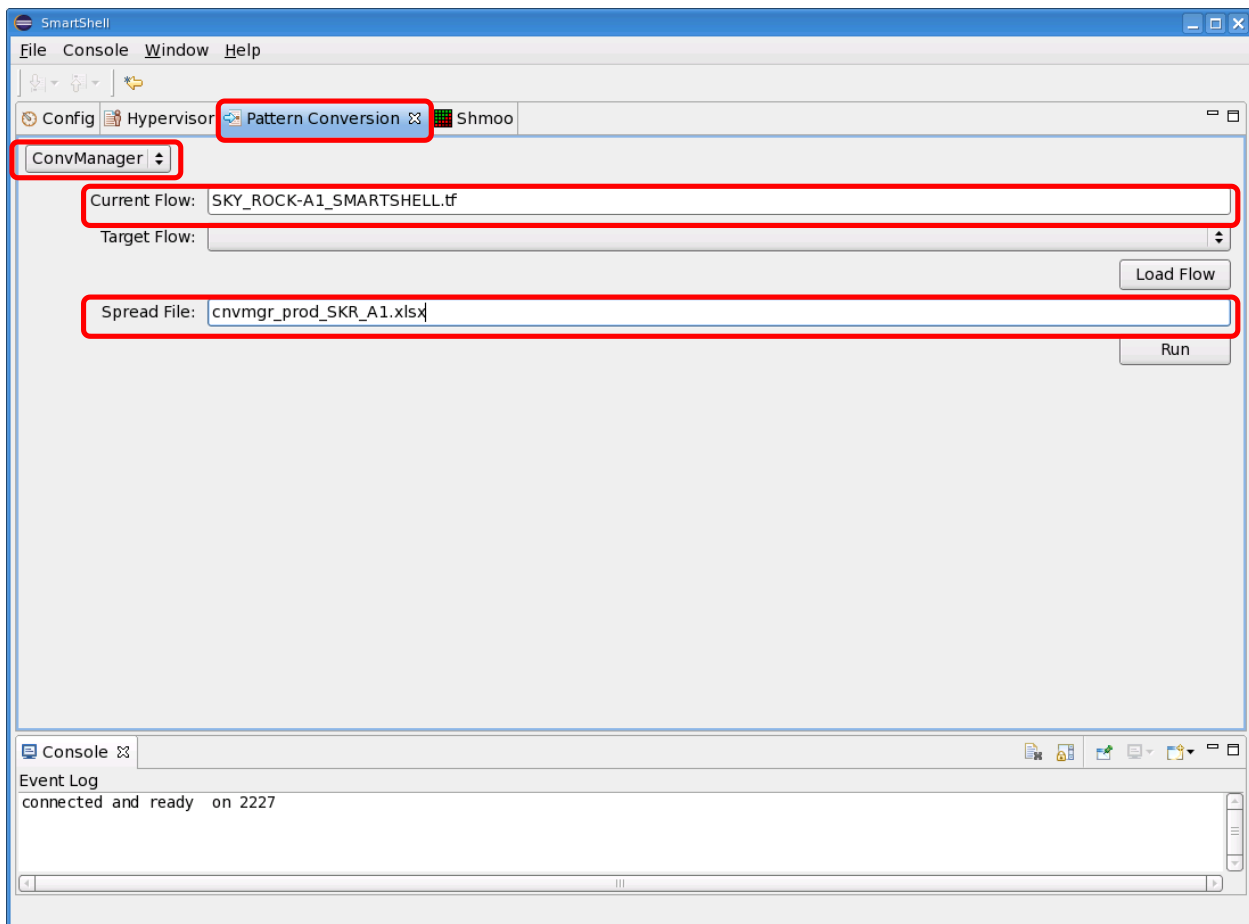


Fig. 18: SmartShell Client GUI Window – Pattern Conversion View

The converted patterns are added to the test program and new test suites are added to a temporary testflow shown as `__tmp_<testflow>` in the SmarTest Work Center.

If the new pattern has the same name as the existing pattern, the existing pattern will be replaced. If the new test suite has the same name as those in the current testflow, the test suite will be appended to the testflow with an index number as `<testsuite>_n` where `n` is incremented.

9.7 Shmoo View

Shmoo is a SmartShell Client GUI feature that do shmoo debug with the specified spec variables.

Procedure

1. Open the SmartShell Client GUI.
2. Click the **Config** tab.
3. Setup **host** and **serverport**.
4. Click the **Connect** button to connect to the SmartShell server.
5. Click the **Shmoo** tab.
6. Click the **Refresh** button.
7. Select **Execution Type** as **Pattern**, **Flow**, or **Suite**.
8. Select the pattern, flow, or suite according to the option you selected in step 7.
9. Select timing specification.
10. Select level specification.
11. Define the spec variables in the setup table by clicking the x-axis and y-axis.
12. Click the **Run** button.

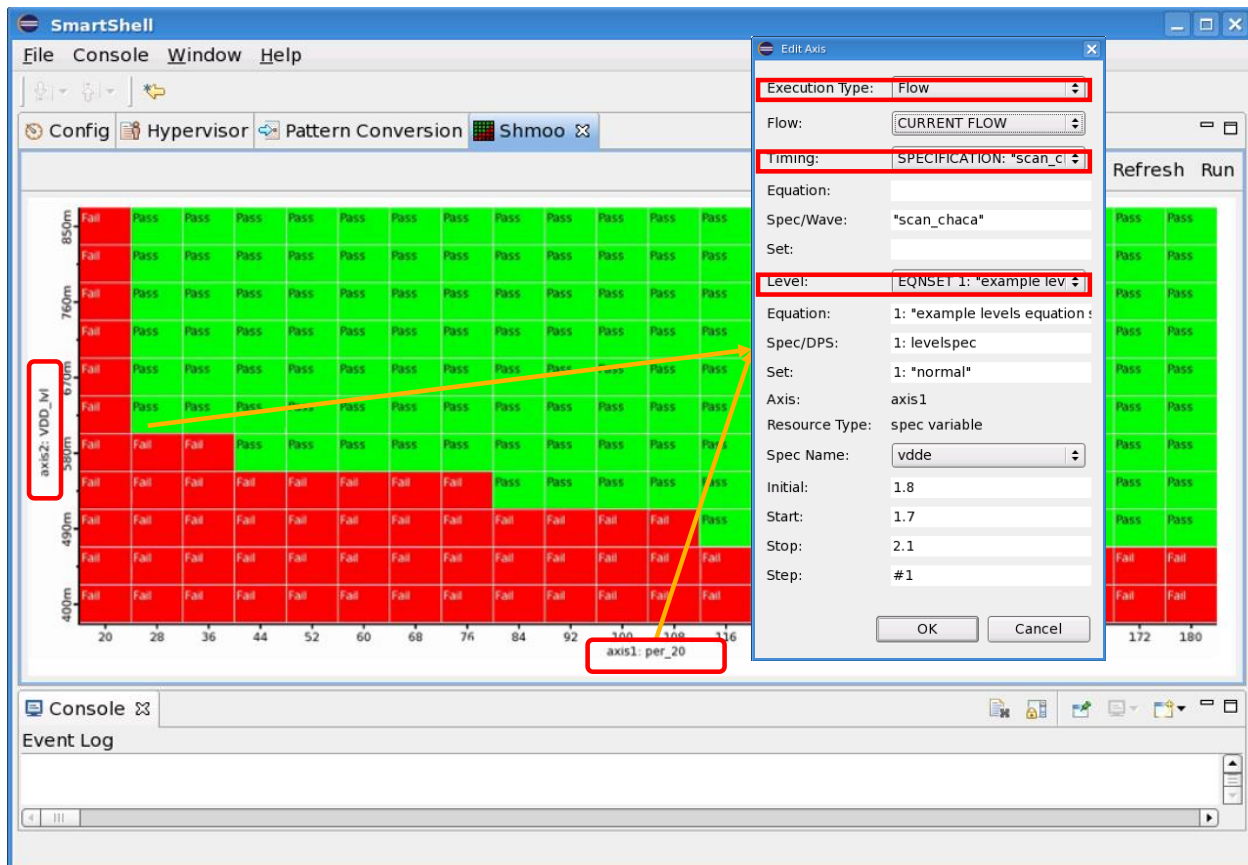


Fig. 19: SmartShell Client GUI Window – Shmoo View